

Variable and Method Flow

Actual variable names, transformations, and method logic across Day 7B notebooks

Learning Day 7B - Classification Missing Coverage Bundle

1. Purpose

This document explains how variables, methods, and data objects move through the three missing Learning Day 7B notebooks. It uses the actual simple variable names used in the notebook workflow.

2. Global Dataset Flow

Step	Variable / Object	Created From	Used For
Load data	data	Train.csv from data/customer_segmentation	Full customer segmentation dataset.
Create features	features	data without ID and Segmentation	Input columns for modelling.
Create target	target	data[Segmentation]	Correct customer segment labels.
Train/test split	train_features, test_features, train_target, test_target	features and target using train_test_split	Separate learning and final testing data.

3. Notebook 1 Variable Flow

Variable	Meaning	Source / Logic	Output / Next Use
classification_types	Table explaining binary, multiclass, multilabel.	Manual learning table.	Documentation and concept summary.
segment_summary	Target count and percentage by segment.	target.value_counts()	Target distribution interpretation.
split_balance	Train/test class balance comparison.	train_target and test_target percentages.	Confirms stratified split.
age_table	Average age by segment.	data.groupby(Segmentation)[Age].mean()	EDA interpretation.
spending_percentage_table	Spending score distribution by segment.	pd.crosstab(... normalize="index")	Spending pattern analysis.
var1_percentage_table	Var_1 category distribution by segment.	pd.crosstab(... normalize="index")	Anonymous category pattern analysis.

4. Notebook 2 Method Flow

Function / Object	Purpose	Input	Output
calculate_gini()	Measure how mixed a group is.	segment_values	Gini value.
calculate_weighted_gini()	Compare split quality using Gini.	data, split_column, target_column	Weighted Gini after split.
calculate_entropy()	Measure confusion in a group.	segment_values	Entropy value.
calculate_weighted_entropy()	Calculate confusion after split.	data, split_column, target_column	Weighted entropy.
information_gain_table	Compare split usefulness.	Entropy before and after split.	Information Gain values.

Function / Object	Purpose	Input	Output
bias_table	Show Information Gain bias.	customer_id vs age_group example.	Explains memorization risk.
advanced_pruning_table	Summarize pruning concepts.	Manual theory table.	Final pruning concept reference.

5. Notebook 3 KNN Variable Flow

Variable / Object	Meaning	Created From	Used For
numeric_columns	Numeric feature names.	features.select_dtypes()	Numeric preprocessing.
categorical_columns	Categorical feature names.	features.select_dtypes()	Categorical preprocessing.
numeric_preprocessing	Median imputation + scaling pipeline.	SimpleImputer + StandardScaler	Clean numeric columns.
categorical_preprocessing	Most frequent imputation + one-hot encoding.	SimpleImputer + OneHotEncoder	Clean categorical columns.
preprocessing	Combined ColumnTransformer.	numeric_preprocessing + categorical_preprocessing	Applies correct logic to each column type.
train_features_processed	Clean numeric training matrix.	preprocessing.fit_transform(train_features)	KNN training input.
test_features_processed	Clean numeric test matrix.	preprocessing.transform(test_features)	KNN testing input.
knn_model	Baseline KNN model.	KNeighborsClassifier(n_neighbors=5)	Baseline training and prediction.
test_predictions	KNN predictions for unseen test customers.	knn_model.predict(test_features_processed)	Accuracy, report, confusion matrix.
k_results_table	Results for different k values.	Loop over k_values.	Select best k.
voting_results_table	Uniform vs distance voting results.	KNN with weights="uniform" and weights="distance".	Select best voting method.

6. KNN Model Pipeline Flow

Order	Action	Object / Method	Why It Matters
1	Load data	pd.read_csv(train_file)	Creates data.
2	Separate input and target	features, target	Keeps answers separate from inputs.
3	Split data	train_test_split(..., stratify=target)	Creates honest test evaluation.
4	Preprocess training data	preprocessing.fit_transform(train_features)	Learns cleaning/scaling only from training data.
5	Preprocess test data	preprocessing.transform(test_features)	Avoids leakage from test data.
6	Train KNN	KNeighborsClassifier().fit()	Stores training examples for neighbor comparison.
7	Predict	model.predict(test_features_processed)	Creates predicted segments.

Order	Action	Object / Method	Why It Matters
8	Evaluate	accuracy_score, classification_report, confusion_matrix	Understands model quality.
9	Tune k	Loop over k_values	Finds better generalization.
10	Compare voting	weights uniform vs distance	Checks vote strategy.

7. Key Rule: Avoid Data Leakage

Preprocessing must be fitted only on the training data. Use `fit_transform` for `train_features` and `transform` for `test_features`. This prevents test information from leaking into the training process.